Reward Shaping Ablation Study

From Local Penalty to Episode-Level Savings

Pulak Mehrotra

April 2026

Contents

This document defines six reward formulations as a structured ablation study, progressing from the narrowest possible signal (single worst MPD) to the true episode-level objective augmented with optimal-gap shaping. Each level expands the scope of information the agent receives, and the ablation is designed to reveal which level is necessary and sufficient to beat greedy.

Sections 3–5 establish the three conceptual building blocks: the topology constraints that limit action scope (MPD reachability), the lower bound that anchors our shaping (optimal gap), and the literature-derived techniques that improve any base reward (PBRS, sub-episodes, counterfactual baselines). Section 6 presents the six concrete formulations.

1 Background and Motivation

The agent distributes each arriving VM's memory request across CXL-connected Memory Pool Devices (MPDs) in a bipartite host–MPD topology. The objective is to minimise the peak per-MPD load over the full trace window, which directly determines pooling savings:

$$\text{pooling\_savings} = 1 - \frac{\max_j c_j^{\text{peak}} \cdot m}{D_{\text{pod}}} \tag{1}$$

A 1–2% improvement over greedy is considered strong.

Runs 5–7 revealed three failure modes that motivate this ablation study:

Run	Reward	Failure mode	Peak savings
6	"current" (Δpeak + variance)	Policy collapsed to 1–2 MPDs; entropy $\to 0$	-0.62 (harmful)
5	Variant A (local projected peak)	Savings plateau; entropy collapse by 2M steps	$\sim$1.3% (flat)
7	Variant A (same, different hyperparams)	Same plateau; Q-values rising without improvement	$\sim$1.2% (flat)

The common thread is **proxy-objective misalignment**: per-step rewards optimise a quantity (instantaneous peak change) that diverges from the true objective (episode-level pooling savings) over 7,000+ allocation decisions. The Noisy Future Knowledge analysis [3] confirms that 5% per-step prediction error compounds over 14 days to produce worse-than-greedy outcomes.

Ablation hypothesis. Progressively widening the reward scope—from single MPD, to reachable neighbourhood, to global pod, to episode-level—will reveal which level of information is necessary and sufficient. Each level adds exactly one dimension of information relative to the previous.

2 Notation

Symbol	Definition
m	number of MPDs in the pod
$N(i)$	MPDs reachable from host i
$H(j)$	hosts connected to MPD j
$c_j(t)$	current allocated load on MPD j (GB)
$D_j(t, W)$	time-weighted departure relief on MPD j over horizon W
$S_j(t)$	sticky load on MPD j (load expected to remain after departures)
x	arriving VM memory request (GB)
$\mathbf{w}$	allocation weight vector across reachable MPDs
W	lookahead window (timesteps)
D_{pod}	total pod DRAM capacity (GB)
$\hat{c}_j(t)$	projected normalized load before placing the new VM
$\hat{c}_j^+(t)$	projected normalized load after placing the new VM
$c_{\max}^{\text{post}}$	$\max_j c_j$ after allocation ($\star$)
$t^*(s)$	optimal achievable peak via Dinic solver ($\star$)
δ_t	optimality gap ($\star$)
$\Phi(s)$	potential function for PBRS ($\star$)
PS	episode-level pooling savings ($\star$)

Core load projection definitions (self-contained). For each MPD j, define post-placement load and normalized projected loads as

$$c_j^{\text{post}}(t) = c_j(t) + x\, w_j, \tag{2}$$

$$\hat{c}_j(t) = \frac{c_j(t) - D_j(t, W)}{D_{\text{pod}}}, \tag{3}$$

$$\hat{c}_j^+(t) = \frac{c_j^{\text{post}}(t) - D_j(t, W)}{D_{\text{pod}}}. \tag{4}$$

3 MPD Reachability

3.1 Topology and Reachability

The pod is a bipartite graph $\mathcal{G} = (\mathcal{H}, \mathcal{M}, \mathcal{E})$ where $\mathcal{H}$ is the set of hosts, $\mathcal{M}$ the set of MPDs, and $\mathcal{E}$ the set of CXL links. Host i can allocate VM memory *only* to its reachable set:

$$N(i) = \{j \in \mathcal{M} : (i, j) \in \mathcal{E}\}. \tag{5}$$

Symmetrically, MPD j receives allocations only from:

$$H(j) = \{i \in \mathcal{H} : (i, j) \in \mathcal{E}\}. \tag{6}$$

AG16x6 example. The default topology (`AG16x6_expander_quads_r5_sym_fixed`) has 16 hosts and a fan-out of up to $d_{\max} = 8$ MPDs per host. Not all hosts share the same reachable set, and many MPDs are shared across multiple hosts ($|H(j)| > 1$). This asymmetry means a uniform reward scope is inappropriate: the information available to the agent varies by host position in the graph.

3.2 Reachability and the Reward Scope Problem

At each step t, the agent receives host i_t's VM request and selects weights $\mathbf{w}_t$ over $N(i_t)$. The action strictly cannot affect any MPD outside $N(i_t)$. Yet the true objective, $\max_j c_j^{\text{peak}}$, is over *all* m MPDs.

This creates a fundamental tension: the reward must evaluate local actions against a global criterion. Three design responses are possible:

1. **Ignore the gap** — reward only the local effect (R1, R2). Simple but proxy-misaligned.
2. **Add a global proxy term** — penalise unreachable hot MPDs as context (R3). Widens scope but introduces action-independent noise.
3. **Evaluate at episode end** — collapse all local decisions into one terminal signal (R4). Fully aligned but sparse.

The ablation ladder tests whether each response is necessary and sufficient, or whether the credit-assignment cost of R4 dominates the alignment benefit.

3.3 Shared MPDs and Inter-Host Interference

When $|H(j)| > 1$, MPD j is a shared resource. If host i_1 fills it near capacity, host i_2 (which also has $j \in N(i_2)$) loses effective allocation headroom at future steps. A reward that observes only $N(i_t)$ at step t cannot detect this cross-host interference until it manifests as a load spike—by which point the damage is done.

The R2 formulation captures the direct post-placement effect on all $j \in N(i_t)$, including shared MPDs connected to the current host. What it misses is the *indirect* effect: that filling a shared MPD constrains a future host's choices. R3 and R4 address this at different levels of fidelity, while the optimal gap (Section 4) provides a model-free lower bound on how much headroom remains globally.

4 Optimal Gap

The codebase includes an exact per-snapshot solver (`octopus/optimal.py`) that computes the minimum achievable peak MPD load $t^*(s_t)$ for a given vector of host demands and topology. It uses binary search over a Dinic max-flow feasibility check. This scalar is a lower bound on any policy's peak load at the current state—the best any allocator could achieve with full knowledge of instantaneous demands but no knowledge of future arrivals.

4.1 Optimality-Gap Penalty

Define the normalised optimality gap after the current allocation:

$$\boxed{\delta_t \;=\; \frac{c_{\max}^{\text{post}} \;-\; t^*(s_t)}{D_{\text{pod}}}} \tag{7}$$

$\delta_t = 0$ means the agent's placement is as good as optimal given current demands. $\delta_t > 0$ quantifies wasted peak capacity. This can be added as a shaping term to any base reward R_k:

$$r'_t \;=\; R_k(t) \;-\; \beta\,\delta_t, \qquad \beta \in [0.1,\ 0.5] \tag{8}$$

Unlike R1–R3, this term is grounded in a provably tight lower bound rather than a heuristic, and it is sensitive to every allocation decision (not just those that change the global worst MPD).

4.2 PBRS-Compatible Version

To guarantee the shaped reward does not change the set of optimal policies (Section 5.1), define the potential:

$$\Phi_{\text{opt}}(s) \;=\; -\,\frac{t^*(s)}{D_{\text{pod}}} \tag{9}$$

Then $F(s, s') = \gamma\,\Phi_{\text{opt}}(s') - \Phi_{\text{opt}}(s)$ rewards reductions in the optimal lower bound itself. Combined with the sparse terminal reward (R4), this is the safest dense signal available: it pushes the policy toward states where the optimally achievable peak is low, without encoding assumptions about departures or future arrivals.

4.3 Implementation Cost

`find_optimal` runs a max-flow per call. At 1,000 steps/episode × 32 envs, calling it every step would dominate wall-clock time. Practical options:

- Call every $k = 10$–50 steps; hold the last value constant between calls (stale-optimal shaping).
- Call only at episode end to compute a terminal potential bonus.
- Pre-compute t^* offline for a representative grid of demand vectors and use nearest-neighbour lookup at training time.

5 Literature-Based Reward Modifications

The base rewards R1–R4 (Section 6) can each be improved by layering principled modifications drawn from RL theory and systems RL literature. This section treats each modification as a composable module that is independent of the base reward choice.

5.1 Potential-Based Reward Shaping

Ng, Harada, and Russell [1] proved that an agent's set of optimal policies is preserved *if and only if* shaping rewards take the form:

$$F(s, s') \;=\; \gamma\,\Phi(s') \;-\; \Phi(s) \tag{10}$$

where $\Phi : \mathcal{S} \to \mathbb{R}$ is an arbitrary potential function. Any other dense reward risks inducing a suboptimal policy relative to the true objective.

R1–R3 are not in this form: they encode transition information that cannot be expressed as a potential difference. This is acceptable for an ablation (we want to test their empirical value), but it means R1–R3 can in principle game the proxy. R4 avoids this by using the true objective, but requires dense shaping to be learnable at ∼7,000-step horizons.

Candidate potentials for this domain.

1. **Simple:** $\Phi(s) = -\max_j c_j(t)/D_{\text{pod}}$. Cheap (one max over m values per step). Rewards peak reductions in a policy-invariant way.
2. **Tight:** $\Phi(s) = -t^*(s)/D_{\text{pod}}$. Uses the Dinic solver (Section 4). Rewards states where the optimal achievable peak is low. Tighter but more expensive.

Both potentials are non-positive and approach zero as the system empties, so $F(s, s') > 0$ when a placement reduces the relevant peak—the direction we want.

5.2 Sub-Episode Decomposition

Mao et al. [4] (Pensieve) and [5] (Decima) both demonstrate that sparse task-aligned rewards with shorter horizons outperform dense proxy rewards with long horizons. Pensieve uses per-chunk rewards; Decima uses job-completion rewards rather than per-step queue-length penalties.

Applied here: the 14-day trace produces episodes of ∼7,000 steps. Breaking the trace at fixed 1–2 day intervals (288–576 steps at 5-minute resolution) turns R4 from one terminal signal per 7,000 steps into one signal per 300–600 steps—a 10× improvement in credit-assignment density while preserving true-objective alignment.

Implementation. At each sub-episode boundary:

1. Compute sub-episode pooling savings from the peak MPD load observed within that window.
2. Return the sub-episode savings as a terminal reward.
3. Reset the peak tracker but **not** the MPD loads—physical state carries over; only the measurement window resets.

5.3 Counterfactual Difference Rewards

Wolpert and Tumer [6] introduced difference rewards to isolate an individual agent's contribution to the global objective by comparing the actual outcome to a counterfactual baseline action a_{base}:

$$D(s, a) \;=\; G(s^a) - G(s^{a_{\text{base}}}) \tag{11}$$

where $G(s) = -\max_j \hat{c}_j(s)$ is the global congestion score and a_{base} is a cheap reference (e.g. greedy or uniform split).

This is most relevant for R3 (Section 6.3), where the global term $\max_{j \notin N(i)} \hat{c}_j$ is action-independent: it penalises global congestion regardless of what the current agent does. Replacing it with a difference reward makes the global term action-dependent and eliminates the moving-baseline problem that inflates critic variance.

5.4 Dynamic λ Scheduling

R3 introduces λ to weight the global term against the local term. A fixed λ is problematic: the right trade-off changes during training, and the topology does not support uniform load. Three alternatives:

Approach	Expected behaviour	Complexity
Remove entirely (use R2)	Eliminates critic noise from action-independent global term	Simplest
Dynamic scheduling	Start $\lambda = 0$ (pure local); increase as policy stabilises via bandit over validation pooling savings	Moderate; requires scheduler
Lexicographic	Optimise local peak first; use global only as tiebreaker when local peaks are equal	Moderate; two-head critic or constraint

For the ablation, run R3 both with and without λ to isolate its contribution. If R2 + PBRS + R4 terminal matches or exceeds R3, then λ adds complexity without value and should be dropped.

6 Ablation Formulations

Six formulations, each adding exactly one dimension of information or one design correction over the previous. R1–R4 form a clean scope progression from narrowest signal to true objective. The final two variants layer the modifications from Section 5 onto the objective-aligned base.

6.1 R1: Single MPD Reward

Definition. Penalise only the single worst MPD after placement.

$$\boxed{R_1(t) \;=\; -\,\frac{c_{j^*}^{\text{post}}(t)}{D_{\text{pod}}}, \qquad j^* = \arg\max_j \, c_j^{\text{post}}(t)} \tag{12}$$

Rationale. The simplest possible signal: how bad is the single worst MPD right now? No neighbourhood context, no departure awareness, no global view. Serves as the controlled floor of the ablation—if any other variant cannot beat R1, it is adding noise rather than signal.

Unlike Run 6's "current" reward, R1 omits the variance term and fair-share normalisation, isolating the effect of scope (single vs. group) from the effect of mixing incommensurable objectives (peak vs. variance).

Signal properties.

- **Density:** Dense. **Scope:** Single global worst MPD. **Range:** $(-1,\ 0]$. **Departure-aware:** No.

Expected failure mode. When the allocation does not land on the current worst MPD *and* does not create a new worst, the reward is identical regardless of which MPD is chosen. Policy gradient is zero with respect to action choice in these cases—the same mechanism that caused Run 6's entropy collapse.

6.2 R2: Localized Group Reward

Definition. Penalise the worst projected load across all MPDs reachable from the current host.

$$\boxed{R_2(t) \;=\; -\max_{j \,\in\, N(i)} \hat{c}_j^{+}(t)} \tag{13}$$

Rationale. R2 improves on R1 in two ways. First, all reachable MPDs contribute: the agent receives distinct signals for placing on a lightly-loaded vs. heavily-loaded reachable MPD, even if neither is the global worst. Second, the projected load subtracts departure relief D_j, distinguishing "full but draining soon" from "full and staying full." This departure awareness is the most important design improvement over R1.

Signal properties.

- **Density:** Dense. **Scope:** All $j \in N(i)$ (typically $d_{\max} = 8$). **Range:** $(-1,\ 0]$. **Departure-aware:** Yes.

Expected failure mode. R2 is blind to unreachable MPDs. A neighbourhood where all MPDs are at 60% looks the same as one where one MPD is at 60% and the rest are empty—R2 loses distributional information. Cross-host interference on shared MPDs (Section 3) is also invisible until it manifests as a spike.

6.3 R3: Global Group Reward

Definition. Penalise the worst projected load locally, plus a weighted penalty for the worst unreachable MPD.

$$\boxed{R_3(t) \;=\; -\max_{j \in N(i)} \hat{c}_j^{+}(t) \;-\; \lambda \cdot \max_{j \notin N(i)} \hat{c}_j(t)} \tag{14}$$

Rationale. R3 tests whether the agent benefits from seeing global pod state beyond its neighbourhood. The second term penalises hot unreachable MPDs, signalling that neighbouring hosts are under stress. Even though the current host cannot directly relieve that stress, the value function may learn to associate global stress with worse long-term outcomes.

Action-dependent variants. The baseline R3 (Eq. 14) suffers from an action-independent global term. Applying the Wolpert–Tumer difference reward (Section 5.3) yields action-dependent alternatives:

$$\textbf{R3-C:} \quad R_{3\mathrm{C}}(t) = -\max_{j \in N(i)} \hat{c}_j^{+}(t) - \lambda\big(G(s_{t+1}) - G(s_t)\big), \tag{15}$$

$$\textbf{R3-D:} \quad R_{3\mathrm{D}}(t) = R_2(t) + \lambda\big(G(s_{t+1}^{a_t}) - G(s_{t+1}^{a_{\text{base}}})\big), \tag{16}$$

where $G(s) = -\max_j \hat{c}_j(s)$ and a_{base} is a greedy or uniform-split baseline. R3-D provides the highest-quality credit assignment for global effect but requires an extra rollout per step.

Signal properties.

- **Density:** Dense. **Scope:** All m MPDs. **Range:** $(-(1{+}\lambda),\ 0]$. **Departure-aware:** Yes (local term and $\hat{c}_j$ for unreachable MPDs).

Expected failure mode. R3 and R3-C still suffer from partial action independence; the global component can become a moving baseline that increases critic variance without improving action ranking. R3-D fixes this in principle but introduces non-trivial compute overhead and reward-scale sensitivity in the choice of λ.

6.4 R4: Episode-Level Reward

Definition. Zero intermediate reward; true pooling savings at episode end.

$$\boxed{R_4(t) \;=\; \begin{cases} 0 & t < T \\ \mathrm{PS}(\text{episode}) & t = T \end{cases}} \tag{17}$$

Rationale. R4 is the only base formulation that directly optimises the true objective. No proxy, no λ, no intermediate approximation. Every distortion introduced by R1–R3 is eliminated. The agent receives a single scalar at episode end that is exactly the quantity we want to maximise.

Signal properties.

- **Density:** Sparse ($\sim$1 signal per 7,000 steps). **Scope:** Global and temporal (captures all placements over the full trace). **Range:** $(-1,\ 1)$ in practice. **Departure-aware:** Not directly (terminal reward reflects actual peaks, not predicted ones).

Expected failure mode. Credit assignment. With 7,000 steps and one terminal reward, the critic must learn which of the 7,000 decisions mattered. SAC's replay buffer and Bellman bootstrapping can propagate this signal backward, but convergence is slow without dense shaping. This motivates the next two variants.

Implementation note. Episode-level pooling savings is already computed in the codebase (`pooling_ratio` / `pooling_savings` in the eval callback). It needs to be wired as the terminal reward rather than a separate logging metric.

6.5 R4 + PBRS + Sub-Episodes

Definition. Episode-level terminal reward with dense potential-based shaping and a shorter sub-episode horizon for credit assignment.

$$\boxed{r_t \;=\; \begin{cases} \gamma\,\Phi(s_{t+1}) - \Phi(s_t) & t \text{ not a sub-episode boundary} \\ \mathrm{PS}_{\mathrm{sub}}(t) + \gamma\,\Phi(s_{t+1}) - \Phi(s_t) & t \text{ a sub-episode boundary} \end{cases}} \tag{18}$$

where $\Phi(s) = -\max_j c_j/D_{\mathrm{pod}}$ (simple potential) or $\Phi(s) = -t^*(s)/D_{\mathrm{pod}}$ (tight potential), and $\mathrm{PS}_{\mathrm{sub}}(t)$ is the pooling savings over the current sub-episode window (1–2 days $\approx$ 288–576 steps).

Rationale. This variant stacks the two literature-based improvements from Sections 5.1 and 5.2. PBRS (Ng et al.) provides dense per-step signal without distorting the optimal policy. Sub-episode decomposition (Pensieve/Decima) shortens the credit-assignment horizon from $\sim$7,000 steps to $\sim$300–600 steps. Together they address the sole failure mode of R4 (sparse signal) while preserving full objective alignment.

Signal properties.

- **Density:** Dense (per-step PBRS shaping) plus terminal at each sub-episode boundary.
- **Scope:** Global and temporal (terminal captures all placements in each sub-episode window).
- **Objective-aligned:** Yes—PBRS preserves the optimal policy set; sub-episode savings is a true partial objective.

Expected failure mode. Added implementation complexity at sub-episode boundaries (peak tracker reset without MPD state reset). PBRS and sub-episodes may be partially redundant: if PBRS alone is sufficient, sub-episodes add complexity without benefit. The ablation should test R4+PBRS alone and the full stack separately to separate their contributions.

6.6 Best Variant + Optimal Gap Shaping

Definition. The best-performing variant from the preceding five, augmented with the optimality-gap penalty (Section 4). Plain penalty form:

$$\boxed{r_t^+ \;=\; r_t^{\text{best}} \;-\; \beta\,\delta_t, \qquad \delta_t = \frac{c_{\max}^{\text{post}} - t^*(s_t)}{D_{\text{pod}}}, \quad \beta \in [0.1,\ 0.5]} \tag{19}$$

PBRS-compatible form using the tight potential:

$$r_t^+ \;=\; r_t^{\text{best}} \;+\; \gamma\,\Phi_{\text{opt}}(s_{t+1}) - \Phi_{\text{opt}}(s_t), \qquad \Phi_{\text{opt}}(s) = -\,\frac{t^*(s)}{D_{\text{pod}}} \tag{20}$$

Rationale. The optimal gap term is the only shaping signal grounded in a provably tight lower bound (Dinic max-flow). It rewards the agent for placing VMs close to the theoretical optimum given current state, independent of future arrivals. Applied to whichever base variant already achieves the best objective alignment, it provides an additional per-step calibration signal at the cost of one max-flow solve per call.

The benefit is largest on R1 and R2 (weakest signal; δ_t adds a lower-bound reference they completely lack) and smallest on R4 and R4+PBRS (already optimising the true objective).

When to evaluate. Optimal-gap shaping should be the last variant tested, after the five-variant primary sweep is complete. The computational overhead is non-trivial and only justified once the strongest objective-aligned base is identified.

Signal properties.

- **Density:** Dense (or stale-optimal at every k steps). **Scope:** Global (Dinic solver sees full topology). **Objective-aligned:** Yes (PBRS-compatible form).

7 Experimental Plan

7.1 Ablation Summary

Variant	Scope	Dense?	Objective-aligned?	Key risk
R1	Single worst MPD	Yes	No	Zero gradient for non-worst placements
R2	Reachable-MPD group	Yes	No	Blind to shared-MPD congestion
R3	Full pod (proxy)	Yes	No	Action-independent global term
R4	Full episode	No	Yes	7,000-step credit assignment
R4 + PBRS + Sub	Full episode	Yes	Yes	Boundary complexity; possible PBRS/sub redundancy
Best + Opt. gap	Full pod + tight lb	Yes	Yes	Compute cost of Dinic per step

7.2 Run Order

1. **R1** — Establish the controlled floor. Cheapest run and cleanest negative control.
2. **R2** — Measure how much local projected-load awareness buys over R1.
3. **R3** — Test whether global context (action-independent) adds value over R2.
4. **R4** — Test whether sparse true-objective signal can learn at all without shaping.
5. **R4 + PBRS + Sub-Episodes** — Test whether objective alignment plus improved credit assignment is the winning combination.
6. **Best + Optimal Gap** — Only after the winner of the above five is clearly established; adds non-trivial compute.

7.3 Metrics

- **Primary:** `eval/pooling_savings_mean` on held-out trace (10 random pod mappings).
- **Diagnostic:** entropy coefficient (`sac/ent_coef`), critic loss convergence speed, Q-value growth rate.
- **Cost:** wall-clock time per 1M steps.

7.4 Success Criteria

A variant "beats" another if it achieves $> 0.5\%$ higher pooling savings on the held-out eval trace, sustained for at least 3 consecutive eval checkpoints (300k steps). Ties are broken by lower variance across pod mappings.

8 Empirical Results: Reward Ablation on Held-Out Traces

All variants trained for 500k steps with SAC, $n_{\text{envs}} = 32$, multi-trace rotation, augmentation enabled, HOTFIX filter off. Evaluation: 20 pod seeds per trace, topology `AG16x6_expander_quads_r5_sym_fixed`.

8.1 LVL01PrdApp05 (churn-dominated trace)

Greedy allocation is harmful on this trace: pooling ratio > 1 means the policy must allocate CXL memory that exceeds the savings from pooling. All RL variants improve over greedy.

Policy	Pool ratio	Savings	vs. Greedy
Greedy	1.033	−3.30%	—
R2	**1.002**	**−0.20%**	**+3.1** pp
R1	1.008	−0.76%	+2.5 pp
R4	1.009	−0.88%	+2.4 pp
R3	1.010	−0.96%	+2.3 pp
current	1.013	−1.31%	+2.0 pp
R5	1.014	−1.40%	+1.9 pp

Table 1: LVL01 held-out evaluation (20 seeds, $n = 20$ iterations). Lower pool ratio is better. All RL variants beat greedy; R2 is closest to break-even.

8.2 AMS20PrdApp19 (long-lived VM trace)

Greedy performs strongly on this trace (ratio 0.721, savings +27.9%). All RL variants cluster near 6.5% savings — a ≈ 21 pp gap behind greedy. Reward variant makes essentially no difference.

Policy	Pool ratio	Savings	vs. Greedy
Greedy	0.721	+27.9%	—
R1	0.934	+6.6%	−21.2 pp
current	0.934	+6.6%	−21.2 pp
R2	0.935	+6.6%	−21.3 pp
R4	0.934	+6.6%	−21.3 pp
R3	0.936	+6.4%	−21.5 pp
R5	0.937	+6.3%	−21.6 pp

Table 2: AMS20 held-out evaluation (20 seeds, $n = 20$ iterations). All RL variants are statistically indistinguishable from each other and trail greedy by ≈ 21 pp. Reward shaping has no effect on this trace.

8.3 Cross-trace summary

The two held-out traces reveal a generalisation gap rather than a reward-shaping effect. On LVL01 (high churn: 64% of VMs live < 1 h, median lifetime ≈30 min) every RL variant outperforms greedy. On AMS20 (long-lived: 46% of VMs live > 7 days, mean ≈3.8 days) greedy dominates. Reward variant R1–R5 differences are within noise on both traces, suggesting the bottleneck is *generalisation across VM-lifetime distributions*, not the reward formulation.

9 Trace Analysis

9.1 Ghost VMs (data quality)

Both traces contain VMs whose `start_time` and `end_time` fields were never populated during trace loading (they retain the class defaults: `start=2024-01-01`, `end=2020-01-01`). AMS20 has 4,788 such ghost VMs (14.1%); LVL01 has 50,674 (18.7%).

These are **safely filtered** by `_build_events`: the base timestamp for each episode is derived from the real VM population (≈ 2023), so the ghost end-time (2020) maps to a tick far below zero, triggering the `if vm_e < 0: continue` guard before any event is appended. Ghost VMs generate no events and do not corrupt the simulation.

9.2 VM-lifetime distributions across all ten traces

Table 3 characterises the full training-trace pool. AMS20 is the only predominantly long-lived trace: 46% of valid VMs run for more than seven days and the median lifetime is ≈20 h. The remaining nine traces are churn-dominated, with LVL01 being the most extreme (64% of VMs live less than one hour, median ≈30 min).

Trace	Valid VMs	Ghost%	<1h%	>7d%	Med (h)	Mean (d)	AvgConc	Med GB
AMS20	27,159	20.2	22.3	**49.3**	**124.6**	**4.14**	14,053	8
BLA	100,404	22.6	29.8	31.2	6.2	2.79	35,032	8
BN9	138,335	22.4	30.5	21.1	2.8	1.92	33,231	16
DSM08	83,252	19.8	33.8	26.3	2.8	2.56	26,677	7
DUB24	64,493	18.5	24.9	37.5	22.8	3.43	27,629	7
LON23	17,831	24.0	35.1	28.3	2.8	2.47	5,498	14
LVL01	181,558	33.0	**56.7**	10.5	**0.75**	**0.98**	22,334	14
SG2	109,483	22.4	32.7	29.2	2.3	2.52	34,498	14
SYD21	46,496	19.1	26.6	36.5	7.2	3.13	18,181	16
YTO21	58,200	27.5	36.4	30.5	1.3	2.56	18,606	8

Table 3: VM-lifetime statistics across the ten training traces. Ghost% = fraction with unset timestamps (safely filtered before simulation). AMS20 is a clear outlier: median lifetime 5+ days, 49% of VMs survive beyond the 7-day window. LVL01 is the most churn-heavy: median 45 min, 57% of VMs live under 1 hour. The 9-to-1 skew toward churn-dominated traces in the training pool explains the generalisation gap observed on AMS20.

9.3 Training composition and the generalisation gap

Both AMS20 and LVL01 are included in the ten-trace multi-trace training pool (`--multi-trace`). Despite this, the RL policy achieves only $\approx 6.5\%$ savings on AMS20 versus greedy's 27.9%. The gap is not caused by AMS20 being unseen during training.

The structural explanation is that on long-lived traces the greedy heuristic (always assign to the least-loaded MPD) is already near-optimal: once the initial allocation is made it persists for days, leaving little room for a learned policy to improve. On churn-dominated traces, rapid reallocation opportunities arise that greedy cannot exploit, giving the RL policy its advantage.

A lifetime-conditioned oracle baseline (assign short-lived VMs greedily, use optimal placement only for long-lived VMs) was evaluated to quantify how much of the AMS20 gap is structurally recoverable.

9.4 Lifetime-conditioned baseline

The baseline routes each VM arrival based on its residual lifetime: VMs with dealloc_tick − tick $\geq$ 288 (24 h) are allocated to the *most-loaded* accessible MPD (bin-pack / consolidate); shorter-lived VMs use the standard greedy water-fill.

Trace	Greedy savings	Lifetime-cond.	Δ
AMS20	+28.6%	−567%	−596 pp
LVL01	−3.3%	−664%	−661 pp

Table 4: Lifetime-conditioned baseline (24 h threshold, 20 seeds). Anti-greedy allocation for long-lived VMs is catastrophically worse on both traces.

The result confirms a key invariant of the pooling objective: greedy water-fill *minimises the instantaneous peak MPD load* for any single VM arrival, regardless of the VM's lifetime. Anti-greedy (consolidating to the most-loaded MPD) concentrates load and maximises the peak. No causal

strategy that deviates from greedy on individual allocations can improve on greedy's instantaneous optimality.

Implication. The 21 pp gap between RL and greedy on AMS20 is *structurally unrecoverable* by any online policy. For long-lived workloads, each placement decision persists for days and greedy's myopic optimality is also globally optimal—there are no short-term deviations that pay off long-term. The RL agents underperform greedy on AMS20 because they have internalised churn-oriented heuristics (learned from the 9 churn-heavy training traces) that actively hurt when VMs are long-lived. Reward shaping (R1–R5) cannot fix this; the bottleneck is the training distribution, not the reward signal.

References

[1] A. Ng, D. Harada, S. Russell. Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping. In *ICML*, 1999.

[2] Huaicheng Li, Daniel S. Berger, Lisa Hsu, et al. Pond: CXL-Based Memory Pooling Systems for Cloud Platforms. In *ASPLOS*, 2023.

[3] Richard Uzeel. Perfect Future Knowledge. Internal project report, 2026.

[4] H. Mao et al. Real World Performance of Adaptive Bitrate Algorithms (Pensieve). In *ACM SIGCOMM*, 2017.

[5] H. Mao et al. Learning Scheduling Algorithms for Data Processing Clusters (Decima). In *ACM SIGCOMM*, 2019.

[6] D. H. Wolpert, K. Tumer. Collective Intelligence, Data Routing and Braess' Paradox. *Journal of Artificial Intelligence Research*, 16:359–387, 2002.

[7] Improving the Effectiveness of Potential-Based Reward Shaping in Reinforcement Learning. arXiv, 2025.